

School of Informatics

Informatics Project Proposal

Unified Multi-Tenant Inference Runtime for Latency-Critical AI Services on Shared RAN Infrastructure

UID: Redacted for review

Date: April 2026

Tutor: Redacted for review

Supervisor: Redacted for review

I give permission for future Informatics Project Proposal classes to read an anonymous version of this document.

This page does not count towards the page limit.

Abstract

The evolution toward 6G mobile networks has catalyzed the AI-RAN paradigm, envisioning a profound convergence of Radio Access Network (RAN) infrastructure and artificial intelligence workloads. Modern GPU-accelerated base stations frequently exhibit significant, temporally bursty spare compute capacity. The central challenge addressed in this research is how to safely harness this underutilized RAN edge compute for latency-critical AI inference tasks without compromising the stringent Service Level Agreements (SLAs) of primary telecommunication functions. This project proposes the design and evaluation of a cloud-native, multi-tenant inference runtime-control architecture. Adopting a strict “RAN-first” operational philosophy, it uses workload isolation and bounded execution control to dynamically multiplex containerized AI inference into the distributed unit (DU) while preserving network reliability.

Declaration of Own Work

I hereby declare that the work presented in this document is my own. The use of Generative AI to assist in the formatting and structuring of this proposal has been fully acknowledged.

1. Motivation

1.1. Problem Statement

As telecommunications networks move toward 6G, recent AI-RAN work increasingly frames the RAN as more than a communications substrate, namely as a cloud-native, distributed platform that can host both primary Radio Access Network (RAN) functions and selected AI workloads on shared infrastructure [1, 2, 3]. A compelling use case within this convergence is latency-sensitive edge inference, where placing model execution close to the radio path can reduce transport delay and keep demanding AI workloads off user devices [1, 4, 5].

RAN infrastructure is also provisioned for peak demand, which leaves meaningful spare capacity outside worst-case conditions [1, 2]. Recent profiling from *Weaver* makes this point concrete: even in a pessimistic four-DU peak-load setting, about 40% spare compute remains, and production-trace replays show that spare streaming-multiprocessor (SM) capacity can still be around 85% during peak daytime periods across six cell sites from three operators [6]. The same study shows that this headroom is bursty within a site but complementary across sites, which makes it valuable yet difficult to harvest with static sharing policies [6].

The central challenge, then, is safe coexistence between workloads with very different timing contracts. DU processing follows tight slot-level deadlines, while edge inference must meet service latency targets without degrading the primary radio function [1, 6, 4]. Existing GPU-sharing mechanisms expose the trade-off: hardware partitioning such as NVIDIA’s Multi-Instance GPU (MIG) offers strong isolation and is therefore a useful hard-isolation option or comparison baseline, but static partitions are a poor fit for bursty millisecond-scale spare cycles; more elastic sharing can expose more headroom, but makes deadline protection harder to guarantee [6, 5]. In this project, that tension is addressed by treating the RAN-side scheduler as an external authority and designing an inference architecture that consumes only the bursty, lease-bounded capacity it exports, rather than by modifying the RAN control stack itself.

1.2. Research Hypothesis and Objectives

1.2.1. Research Hypothesis

Recent evidence from AI-RAN platforms and device–RAN–cloud inference studies suggests that latency-critical edge inference is constrained less by accelerator presence alone than by whether heterogeneous workloads can be controlled under coupled uplink, compute, and downlink pressure, hard memory limits, and intermittent compute availability [2, 1, 4, 5, 8]. The central systems challenge is therefore to identify the shared runtime mechanisms that make diverse workload families compatible with RAN-edge operation, while preserving the distinct execution boundaries, state semantics, and memory behavior that those families expose.

Therefore, the core **Research Hypothesis** of this proposal is: *A unified multi-tenant RAN-edge inference system that combines workload-aware micro-segmentation, explicit state parking, memory-bounded admission control, and stage-aware runtime control can support heterogeneous latency-critical inference workloads without violating RAN-first service guarantees. We further hypothesize that such a design can maximize useful inference progress within bounded execution windows while remaining stable under reclaim events, strict VRAM limits, and workload-specific phase changes.*

1.2.2. Objectives

To test this hypothesis and realize the revised project scope, this research will pursue five tightly scoped objectives.

1. **Extract a workload-property baseline for RAN-edge inference:** derive a profiling baseline from CORA-style workload classes and recent edge inference evidence, explicitly identifying the natural execution unit, dominant uplink/compute/downlink pressure, persistent state, and safe interruption boundary for representative one-shot and streaming workloads [4, 5].
2. **Identify the shared mechanisms and workload-specific adapters required for unification:** determine which mechanisms must be shared across workloads—for example admission control, micro-segmentation, state parking, memory budgeting, and isolation—and which adapters must remain specific to activation-heavy vision, stateful streaming, or rendering workloads [1, 4, 5].
3. **Define a unifying runtime control abstraction and architecture:** specify how a multi-tenant RAN-edge inference system should represent bounded execution windows, runtime state, memory budgets, and reclaim behavior inside a container-level control and data plane that treats telecom traffic as the first-class tenant [1, 7, 5].
4. **Incorporate radio-resource coupling into runtime decisions and evaluation:** translate the workload baseline into a control logic that reacts to uplink/compute/downlink pressure, intermittent compute availability, and workload-specific phase changes, so that useful work is maximized without breaking RAN timing priorities [4, 8, 5].
5. **Prototype and evaluate the unified architecture in an edge-oriented environment:** implement the proposed mechanisms using a containerized inference stack and quantify whether the resulting system preserves RAN-first guarantees, remains stable under hard VRAM limits and reclaim events, and delivers acceptable latency across heterogeneous edge inference services [1, 8, 5].

1.3. Timeliness, Novelty, and Significance

1.3.1. Timeliness

The timing is favorable because AI-RAN is moving from broad vision to concrete, cloud-native architectural proposals and deployment workflows [1, 2, 3]. At the same time, large-scale RAN experimentation and latency-critical cellular inference are becoming more concrete through public-cloud emulation platforms such as *CHRONOS* and end-to-end coordination frameworks such as *CORA* [8, 4]. Recent device-RAN-cloud inference work further suggests that SLA-aware placement is becoming a practical systems question rather than a purely conceptual one [5].

1.3.2. Novelty and Relation to Prior Work

Prior work addresses important pieces of this problem, but not the full combination required here. *Beyond Connectivity* and related AI-RAN architecture efforts describe the orchestration substrate for shared AI and RAN infrastructure [1, 2, 3]. *Paradrop* demonstrates lightweight multi-tenancy at the extreme edge [7]. *CORA* coordinates uplink, compute, and downlink for latency-critical inference [4]. *Weaver* shows how to expose RAN-safe spare GPU capacity, but

its target workload is foundation model training rather than latency-critical inference [6]. *SLA-Aware Distributed LLM Inference Across Device-RAN-Cloud* further suggests that strict SLAs may require hard isolation or carefully selected model variants, which makes MIG a useful comparison point rather than a universal answer [5].

The novelty of this research lies in combining these strands into one runtime-control problem: determining which inference workloads can share DU-adjacent compute, under what isolation policy, and with what end-to-end latency budget. The project therefore focuses on phase-aware inference profiling, DU-local coexistence control, and multi-tenant orchestration that joins spare-compute harvesting, end-to-end latency coordination, and edge lifecycle management into one unified RAN-edge inference design [6, 4, 7].

1.3.3. Significance

If successful, this research could clarify whether multi-tenant edge inference can be executed natively on RAN infrastructure without degrading network reliability. For network operators, that would provide a blueprint for monetizing otherwise idle GPU cycles without deploying new hardware. For end-users and device manufacturers, it would strengthen the case for low-latency edge support for lightweight, battery-efficient mobile devices. More broadly, the project would contribute a technical foundation for treating AI services as a native component of 6G networks.

1.4. Feasibility

This proposal is feasible because the dissertation scope is deliberately constrained to the inference runtime layer rather than to a full RAN-stack redesign. The project begins from an existing report-derived workload baseline, reuses public AI-RAN and device-RAN-cloud evidence for its design assumptions, and targets a containerized prototype with controlled replay inputs instead of a production deployment. That keeps the implementation burden focused on profiling, control logic, runtime mechanisms, and evaluation artifacts that are realistic within the dissertation timescale.

1.5. Beneficiaries

The primary beneficiaries are AI-RAN and edge-systems researchers who need a clearer account of which inference workloads can safely coexist with RAN functions under strict timing constraints. Network operators and platform engineers would also benefit from a concrete runtime-control blueprint for converting bursty DU-adjacent headroom into bounded AI service capacity without weakening telecom-first protection. A secondary beneficiary group is device and application designers, because a credible RAN-edge inference substrate would broaden the range of low-latency AI services that can be offloaded from resource-constrained endpoints.

2. Background and Related Work

This section provides the foundational context necessary to understand the proposed RAN-edge inference runtime. It first outlines the evolution of cloud-native telecommunications and edge AI, before critically reviewing the existing literature on GPU sharing, edge multi-tenancy, and RAN-aware coexistence control to highlight the specific gaps this research will address.

2.1. Background

2.1.1. Cloud-Native O-RAN Architecture

Historically, cellular network deployments relied on monolithic, proprietary hardware appliances, which reinforced vendor lock-in and rigid operational models. Open RAN (O-RAN) marks a shift toward disaggregated baseband functions and standardized open interfaces [3, 1, 2]. A central part of this design is the functional split, especially Split 7.2, which separates lower-PHY processing at the Radio Unit (O-RU) from the more computationally intensive upper-PHY operations at the Distributed Unit (O-DU) [3, 1, 2].

This shift also moves RAN execution away from fixed-function telecommunications equipment toward a cloud-native operating model [3, 1, 2]. In virtualized RAN (vRAN) deployments, network functions are decoupled from specific hardware and run as containerized microservices on Commercial Off-The-Shelf (COTS) servers that collectively form the O-Cloud [3, 1]. The Service Management and Orchestration (SMO) framework then applies standard cloud-orchestration principles to lifecycle management, scaling, and deployment across that substrate, while supporting multi-vendor interoperability and programmatic control [1, 2]. AI/ML-driven automation extends this model further by coordinating both computational and communication resources [2, 1].

However, realizing this cloud-native flexibility introduces severe execution challenges, primarily due to the stringent, deterministic nature of cellular telecommunications. RAN functions demand substantial compute and network resources. The network operates in a strict time-slotted fashion, with slot durations ranging from hundreds of microseconds to just one millisecond [8]. During a single slot, the system must execute radio resource allocation in the control plane and process user packets in the data plane (e.g., segmentation, header attachment, and retransmissions) [8].

Executing these slot-level tasks on standard cloud infrastructure is demanding. CHRONOS shows that VM interruption and network latency can consume a large share of the slot budget, so faithful cloud execution depends on tight release synchrony and careful control of timing jitter [8]. This timing pressure helps explain why recent O-RAN and AI-RAN work places disaggregated DU/CU software, O-Cloud resources, and accelerator support within a common cloud-native operating model, rather than treating real-time RAN functions as fixed-purpose appliances [3, 1, 2].

2.1.2. The AI-RAN Paradigm

As cellular systems move toward 6G, the RAN is increasingly framed as both a communications system and a distributed compute platform. The AI-RAN literature describes this convergence as a joint evolution of infrastructure, control, and services, with AI capabilities embedded into the RAN stack and coordinated across edge and cloud resources [1, 2, 3].

Recent literature commonly organizes this convergence into three closely related pillars [2, 3]:

- **AI-for-RAN (Network Optimization):** This pillar applies AI to improve internal network functions, including scheduling, beam management, traffic steering, energy saving, and anomaly handling [2, 3]. Here, AI remains subordinate to the primary telecommunications service.
- **AI-on-RAN (Native AI Services):** This pillar treats the RAN and its edge compute continuum as a native platform for AI services, especially distributed learning and low-latency inference placed close to DU/CU resources [2].

- **AI-and-RAN (Infrastructure Multiplexing):** This pillar asks when spare O-Cloud or AI-RAN compute can host secondary AI workloads without breaking primary radio deadlines [2, 6, 1].

This research proposal focuses strictly on the **AI-and-RAN** pillar. Relative to the broader AI-for-RAN literature, it remains at an earlier stage of system design and validation, even though it offers a clear economic reason to reuse AI-RAN infrastructure beyond peak radio demand periods [2, 6, 1].

In practice, that reuse depends on turning spare DU or O-Cloud compute into a controlled edge-service substrate while preserving the primacy of RAN timing and reliability requirements [1, 6, 8]. The central systems question is therefore not whether AI workloads can be injected in the abstract, but how to admit and coordinate only those workloads that remain compatible with underlying radio deadlines [2, 6].

2.1.3. The Anatomy of Latency-Critical Inference Workloads

Latency-critical edge inference is not a single workload class but a family of execution patterns whose feasibility depends on how uplink, compute, and downlink pressure interact across the end-to-end service path [4, 5]. CORA is useful here because it frames inference as a resource-coupled pipeline rather than as a GPU-only problem. For a RAN-edge system, the relevant question is therefore not simply whether an accelerator is present, but what execution unit, persistent state, dominant stage pressure, and interruption boundary each workload family exposes.

One-shot vision workloads such as image segmentation and pose estimation become compute-dominant once inputs have arrived over the uplink; their execution unit is the full inference or a coarse tile, their persistent state is limited to transient activation tensors, and their safest interruption boundary is between inferences or coarse tiles [4]. Super-resolution is similarly compute-dominant but more memory-stressed because large activation maps and upsampling buffers inflate its live state; its practical execution unit is therefore again the full frame or a coarse tile, with no useful persistent state beyond activations that usually must be drained rather than parked mid-pass. Volume-rendering workloads such as NeRF are also compute-dominant, but their natural execution unit is a ray batch and their persistent state is the compact scene or radiance representation, which makes ray-batch boundaries the least disruptive stop point.

Streaming language workloads are phase-split. Prompt ingress creates an uplink/compute-coupled prefill stage. Autoregressive decoding is compute/downlink-coupled and advances one token step at a time, so the execution unit is the token step and the persistent state is the KV cache [5, 11, 10]. Seq2seq translation likewise has an uplink/compute-coupled encoder pass followed by a compute/downlink-coupled decoder phase, so the useful execution unit is the decoder step after an encoder pass and the persistent state is the encoder output plus decoder state. These families therefore differ not only in latency target but in the pressure that dominates them, the state that must survive reclaim, and the boundary at which useful work can be paused. A unified RAN-edge inference system must intersect coarse-boundary one-shot execution, stateful streaming control, hard memory budgeting, and workload-specific stop points within one common runtime framework [4, 5].

2.1.4. Strict RAN Service Level Agreements (SLAs) vs. IT Workloads

Sharing this cloud-native hardware introduces a real clash of scheduling objectives. In standard IT clouds, schedulers can chase fairness or average utilization. In the RAN, CHRONOS shows that VM interruption and network latency can already consume a large fraction of the slot

budget, so release synchrony and timing jitter are first-order constraints rather than implementation details [8]. Any coexistence policy for DU-adjacent GPUs therefore has to preserve a strict “RAN-first” priority model instead of a best-effort cloud policy [8, 6].

2.1.5. The Hardware Dichotomy: Training vs. Inference

Within the AI domain, the workload type matters as much as the accelerator itself. Prior RAN-edge sharing studies suggest that foundation-model training is the friendlier co-located case: LDPC-heavy RAN processing is largely bandwidth-dominated, while training is more compute-heavy and elastic, so spare SM envelopes can be harvested for secondary work without assuming the same resource profile as the radio path [6]. By contrast, CORA and recent device, RAN, and cloud measurements frame inference as an interactive end-to-end service whose feasibility depends on model variant, placement, channel quality, and deadline budget rather than on GPU presence alone [4, 5]. Mechanisms that are adequate for elastic training therefore cannot be transferred directly to latency-critical inference [6, 4, 5].

2.2. Related Work

2.2.1. Hardware-Level GPU Sharing Mechanisms

To realize AI-and-RAN, the GPU must be shared between the RAN DU and the inference stack, but the available mechanism choices mostly expose a trade-off rather than a complete answer. Spatial partitioning such as MIG offers strong isolation and is therefore a useful baseline when strict SLA classes need dedicated slices, yet static partitions are a poor fit for bursty spare compute and can strand capacity that arrives at slot timescales [6, 5]. More elastic temporal sharing can expose more headroom, but existing RAN-aware sharing studies also show that mechanism overhead and sharing jitter remain central concerns for safe reuse [6]. These primitives matter, but they do not by themselves provide an inference-aware runtime-control policy for a shared DU GPU [6, 5].

2.2.2. Edge Multi-Tenancy and Orchestration

Providing AI inference as a multi-tenant service also requires a slow-timescale orchestration layer. Paradrop is a useful precedent here because it shows that lightweight, containerized services can be deployed at the network’s extreme edge with backend orchestration and explicit resource controls [7]. That said, Paradrop targets lightweight edge services on constrained nodes, while AI-RAN proposals extend orchestration toward the O-Cloud and SMO stack for joint RAN and AI resource management [7, 1]. The orchestration lesson is valuable, but it still has to be paired with a fast local execution policy for shared accelerators.

2.2.3. End-to-End Latency Coordination

Inference latency spans uplink transport, edge execution, and downlink return, and CORA is the strongest direct template in this corpus for coordinating those stages [4]. CORA explicitly budgets latency across uplink, compute, and downlink, derives RB demand from I/O size and channel quality, derives SM demand from a speedup model, and performs pre-admission costing with lightweight planner overhead of about 0.18 ms and roughly 112 B of metadata per request [4]. This is strong evidence that end-to-end latency budgeting is feasible, but CORA still treats

compute as a schedulable stage budget rather than as a shared RAN-GPU runtime-control problem [4, 5].

2.2.4. RAN-Aware Co-scheduling

Recent RAN-aware sharing studies come closest to the local coexistence problem by showing that DU-adjacent GPU headroom can be exported in bursty, slot-bounded envelopes for secondary AI work [6]. Their profiling is concrete: even a pessimistic four-DU peak-load setup still leaves about 40% spare SM capacity, and production-trace replays show spare capacity can remain around 85% during peak daytime periods, although that headroom is bursty within sites and complementary across sites [6]. However, the evaluated secondary workload is foundation-model training rather than interactive inference, and the same evidence base warns that LLM decoding is also bandwidth-heavy, which makes it a weaker sharing match with LDPC than training is [6, 5].

2.3. Summary of Research Gap

Taken together, the literature covers the surrounding layers but not the core inference-sharing problem. CHRONOS explains why slot-realistic timing leaves little slack for interference, while Paradrop supplies a pattern for multi-tenant edge orchestration [8, 7]. CORA supplies end-to-end latency budgeting, and RAN-aware sharing studies show how compatible secondary workloads can harvest spare RAN GPU capacity [4, 6]. What is still missing is an inference-specific runtime-control layer for shared RAN GPUs, one that reasons about phase-sensitive inference behavior and admits only those requests whose latency and resource profile remain compatible with slot-level DU deadlines [6, 4, 5].

3. Methodology

This dissertation adopts a report-driven systems methodology targeted at the inference runtime layer of AI-RAN rather than at radio scheduling itself. The methodological problem is to determine how a unified multi-tenant inference system should operate under bounded execution windows, hard memory budgets, and reclaim events while preserving a strict RAN-first priority model. In the later design phases this bounded execution interface is formalized as a **ComputeLease**, but the methodological logic begins from workload properties, control decisions, and implementation artifacts rather than from the contract term alone.

3.1. Methodology or Approach

Phase 1: Profiling and workload-property extraction. The methodological inputs are the CORA workload taxonomy, the report-derived mechanism synthesis, and the RAN-edge timing constraints established earlier in this proposal [4, 5, 8]. The first phase builds a profiling baseline that names, for each workload family, the dominant uplink/compute/downlink pressure, the natural **execution unit**, the persistent **state** that survives across useful work, and the safest cooperative interruption boundary. Concretely, this phase distinguishes coarse one-shot boundaries for activation-heavy vision and super-resolution, token-step or decoder-step boundaries for streaming transformers, and ray-batch boundaries for rendering workloads. The primary artifact is therefore not a generic survey but a workload-property matrix that records execution unit, state model, stage pressure, and preemptibility in a form that can drive later implementation decisions.

Phase 2: Shared-mechanism synthesis and control-abstraction design. The second phase turns the profiling matrix into a set of shared runtime mechanisms. Guided by the report synthesis, the methodology treats lease-aware admission control, micro-segmentation selection, state parking, memory budgeting, isolation policy, and telemetry as the common mechanism set that must span all workloads, while keeping room for workload-specific execution adapters [1, 4, 5]. Here the key research task is to define the unifying control abstraction: what bounded execution windows mean operationally, what information must be observable at runtime, and which **decision variables** the controller may change when conditions deteriorate. The explicit deliverable is an architecture specification that names the control plane, data plane, unit-of-control semantics, reclaim behavior, and the artifact interfaces between them.

Phase 3: Workload-specific adapters and CORA-coupled policy construction. The third phase designs the workload-specific branches that sit underneath the shared control plane. Streaming workloads require adapter logic for KV-cache or encoder/decoder state, pause-at-step boundaries, and state parking under reclaim; one-shot workloads require coarse tiling or per-inference draining policies because their activations make mid-execution interruption expensive; rendering workloads require ray-batch sizing and explicit handling of compact scene state. CORA enters this phase as a real control model rather than as background context: the controller will use stage-aware uplink, compute, and downlink pressure together with runtime **observables** such as per-work-unit runtime, VRAM headroom, time-to-first-token, and recovery latency to make placement, admission, execution-window sizing, degradation, and parking decisions [4, 5]. This is the point at which the proposal makes the intersection logic explicit: the runtime is unified at the level of shared mechanisms, but each workload family keeps its own adapter and stop-point policy.

Phase 4: Prototype implementation, artifact ladder, and evaluation bundle. The fourth phase operationalizes the design through a containerized prototype runtime implemented with PyTorch-based inference components and reproducible edge-oriented experiments. The implementation artifact ladder is explicit: (i) the profiling matrix and workload-property baseline, (ii) the unified architecture and control-abstraction specification, (iii) workload-specific adapter designs, (iv) a prototype runtime with admission, parking, memory-budgeting, and telemetry paths, and (v) an evaluation bundle containing traces, configurations, and results. This phase treats the external resource allocator as an assumed provider of bounded execution windows, not as a subsystem to redesign, so the prototype can be exercised under replayed lease traces, synthetic reclaim events, and heterogeneous workload mixes while remaining faithful to the intended RAN-edge operating conditions [1, 8].

3.2. Expected Outcomes

The first expected outcome is a profiling and analysis baseline that formalizes which workload classes are plausible at the RAN edge, what their execution units and state semantics are, and how stage pressure shifts across uplink, compute, and downlink. This baseline is valuable because it converts a broad AI-and-RAN motivation into an auditable design input for implementation.

The second expected outcome is a concrete architecture for a unified multi-tenant RAN-edge inference runtime. That architecture will define the shared mechanism set, the workload-specific adapters, the control/data-plane split, the runtime observables, and the decision variables used to keep inference compatible with RAN-first constraints.

The third expected outcome is an executable prototype and evaluation bundle showing how the proposed runtime behaves under reclaim, strict VRAM limits, and heterogeneous workload

mixes. Together, these artifacts should provide both a dissertation-scale research contribution and a practical engineering blueprint for future AI-and-RAN deployments.

3.3. Evaluation Criteria

Evaluation will first test **telecom-safe execution**. A configuration will only be considered viable if inference work yields cleanly to reclaim events, respects strict priority for telecommunications traffic, and does not depend on redesigning the external scheduler. For streaming workloads, core metrics include time-to-first-token, per-step service time, and recovery latency under interruption; for one-shot workloads, the primary metric is completion latency relative to the granted execution window [4, 5].

The second evaluation group targets **resource survival under coupled edge constraints**. Here the methodology will measure whether live memory remains below the configured budget, whether parked state can be restored without destabilizing latency, and whether fragmentation or transient-buffer growth causes failure under repeated reclaim. CORA-coupled reasoning is explicit in this stage: when uplink pressure dominates, the system must avoid overcommitting one-shot workloads whose useful progress is input-gated; when compute or downlink pressure dominates, the controller must adapt execution-window sizing, parking aggressiveness, or degradation policy rather than treating all workloads as equivalent [4, 5].

The final evaluation group measures **execution efficiency and control quality**. The key metrics are micro-segmentation overhead, admission precision, useful work completed per execution window, and the stability of runtime decisions under changing channel and resource conditions. These evaluation outputs are intended to show not only that the architecture is safe, but that the unified control logic remains worthwhile once real workload diversity, reclaim volatility, and stage-aware coupling are taken into account.

4. Project Plan and Timeline

This section does not count towards the page limit.

The project is structured as a twelve-week dissertation plan that mirrors the revised methodology. Report-driven profiling and workload-property extraction are completed first so that the rest of the study is grounded in an explicit baseline of execution units, state semantics, and stage pressure rather than in informal assumptions. Architecture synthesis and workload-adaptor design then lead into prototype construction, after which CORA-coupled control integration and evaluation are used to validate the runtime under reclaim, hard memory limits, and heterogeneous workload mixes. Dissertation writing begins before the final experiments conclude so that schedule risk is absorbed through continuous integration of results rather than through a single late write-up phase. If the later control logic proves harder than expected, the dissertation still retains value through the profiling baseline, the unified architecture specification, the adaptor/policy design, and the experimentally validated prototype runtime.

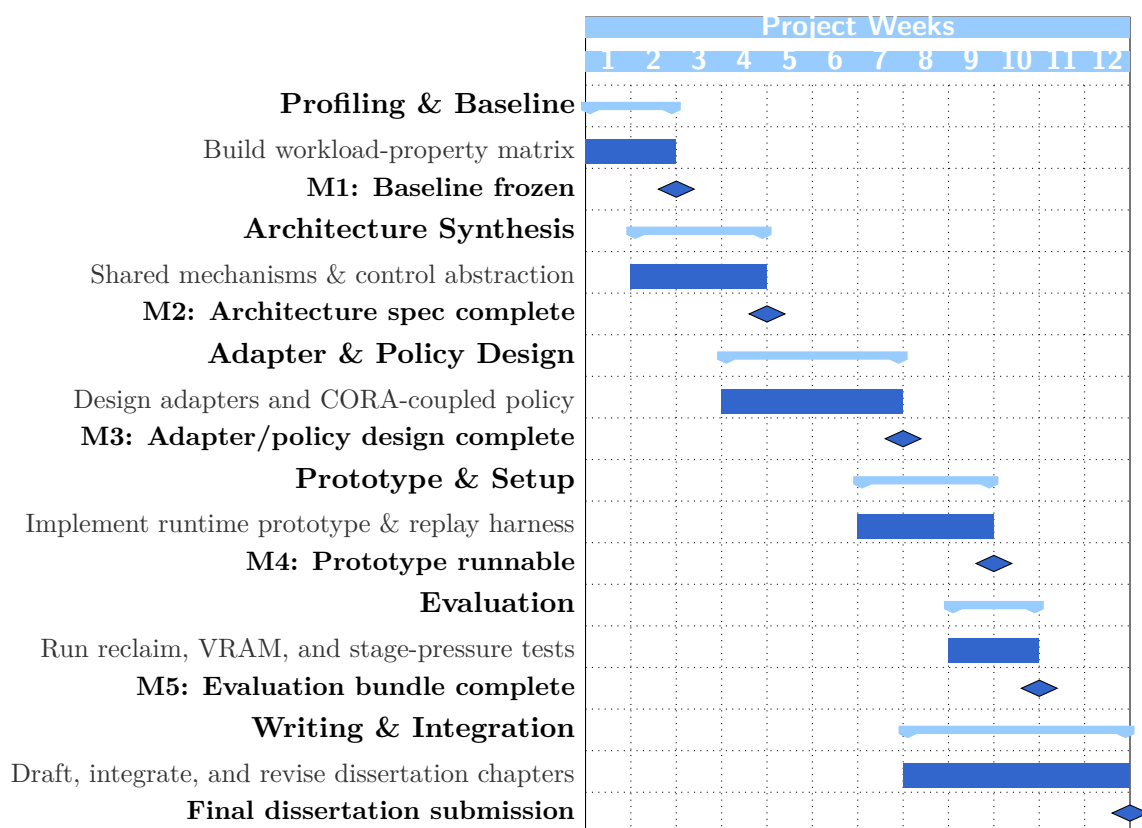


Figure 1: Twelve-week project plan aligned with the report-driven methodology.

References

- [1] Michele Polese, Niloofar Mohamadi, Salvatore D'Oro, Leonardo Bonati, and Tommaso Melodia. Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G. *arXiv preprint arXiv:2507.06911*, 2025.
- [2] Chenyuan Feng, Howard H. Yang, Kun Guo, Wenchao Xia, Chenxi Liu, and Tony Q. S. Quek. AI-RAN: The pathway to future wireless networks. *Journal of Information and Intelligence*, 4(1):5–22, 2026.
- [3] *Proceedings of the 2025 2nd ACM Workshop on Open and AI RAN*, Hong Kong, China, 2025. Association for Computing Machinery. Part of MobiCom '25.
- [4] Sunghyun Jin, Serae Kim, Sangtae Ha, and Kyunghan Lee. End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks. *Proceedings of the ACM on Networking*, 3(CoNEXT4):1–23, 2025. Article 40.
- [5] Hariz Yet, Nguyen Thanh Tam, Mao V. Ngo, Lim Yi Shen, Lin Wei, Jihong Park, Binbin Chen, and Tony Q. S. Quek. SLA-Aware Distributed LLM Inference Across Device-RAN-Cloud. In *IEEE INFOCOM Workshops 2026 (6G AI-RAN 2026)*, 2026. Preprint on arXiv:2602.23722.
- [6] Weaver: Foundation Model Training over AI-RAN Compute Infrastructure, 2026. Local ACM-formatted manuscript dated March 2026.
- [7] Peng Liu, Dale Willis, and Suman Banerjee. Paradrop: Enabling Lightweight Multi-tenancy at the Network's Extreme Edge. In *IEEE/ACM Symposium on Edge Computing (SEC)*, pages 1–13, 2016.
- [8] Ujjwal Pawar, Andrew E. Ferguson, Yuto Takano, Jon Larrea, Xenofon Foukas, Mahesh K. Marina, and Bozidar Radunovic. Towards Scalable and Cost-Effective RAN Emulation Leveraging the Public Cloud. In *Proceedings of the 26th International Workshop on Mobile Computing Systems and Applications*, HotMobile '25, pages 43–48, 2025.
- [9] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gula-vani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. *arXiv preprint arXiv:2403.02310*, 2024.
- [10] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, Santa Clara, CA, 2024. USENIX Association.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint arXiv:2309.06180*, 2023. SOSP 2023.

This section does not count towards the page limit.